



American International University-Bangladesh (AIUB)

**Department of computer science
Faculty of Science & Technology (FST)**

Semester: FALL 2025-26

Course: Data Science

Section : J

Group: A

Project Title:

**Statistical Analysis of Lifestyle and Academic Factors Influencing
Student Mental Health**

Dataset Title: Depression Student Dataset

Supervised By:

Kamrun Naher Koli

Submitted By :

Name	ID
Taharat Muhamud Jabir	22-49037-3
Merin Taj	23-50865-1
Wasimul Bari Rahul	23-50913-1
MD.Hasibul Islam	23-51777-2

Submission Date : 30-11-2025

Introduction

This report analyzes the factors influencing student mental health and academic performance using a Student Mental Health Dataset, focusing specifically on survey records capturing diverse student demographics. The project entailed a comprehensive workflow starting with data preprocessing to clean and standardize the dataset, which included imputing missing values and removing outliers from study hour logs. This was followed by detailed exploratory data analysis to investigate the distribution of key metrics such as depression rates, academic pressure, and sleep regularity. By examining the relationships between numerical attributes, such as daily study hours and financial stress, and categorical features like dietary habits and family history of mental illness, this study aims to uncover the underlying lifestyle and environmental factors that drive student well-being. All data manipulation, statistical computing, and visualization tasks were executed using the R programming language.

Dataset Description

The dataset used in this project is titled "Depression Student Dataset" and contains comprehensive survey information regarding student lifestyles. It comprises multiple rows of respondent data with 11 variables, offering details at both personal and academic levels. The dataset includes several numerical features that quantify student stressors and habits. Key variables include:

- **Academic Pressure** (rated on a scale)
- **Study Hours** (daily duration)
- **Financial Stress** (rated on a scale)
- **Age**

Additionally, the dataset contains structural categorical data that provides qualitative insights into the student's profile. These include:

- **Dietary Habits** (classifying habits as "Healthy", "Moderate", or "Unhealthy")
- **Sleep Duration** (categorized ranges like "5-6 hours" or "More than 8 hours")
- **Depression** (a boolean-style variable indicating "Yes" or "No")
- **Family History of Mental Illness**

Other descriptive variables include **Gender** and **Study Satisfaction**, which provide the necessary context for analyzing trends across different student groups. Overall, this dataset provides a robust mix of lifestyle metrics and psychological indicators, making it suitable for

performing Exploratory Data Analysis (EDA) and studying the correlation between academic rigor and mental health.

Project Implementation

Task-1: Environment Setup and Library Initialization

The first step of the analysis establishes the R working environment by importing specific libraries. These packages provide the necessary functions for data ingestion, cleaning, and statistical visualization, creating the framework for the subsequent analysis of the student mental health dataset.

Code:

```
# Task-1: Setup Libraries  
library(dplyr)  
library(readr)  
library(ggplot2)  
library(corrplot)  
library(tidyr)
```

Output:

```
> # Task-1: Setup Libraries  
> library(dplyr)  
> library(readr)  
> library(ggplot2)  
> library(corrplot)
```

Code description of task-1:

- **Data Import (readr):** This library is utilized to load the Midterm.csv dataset into the R environment. It ensures that the raw survey data is imported correctly and efficiently.
- **Data Preparation (dplyr & tidyr):** These packages are essential for organizing the data. dplyr is used to filter, select, and summarize specific rows, while tidyr handles the imputation of missing values to ensure the dataset is complete before analysis.
- **Visualization (ggplot2):** We employ this library to generate the visual components of the report. It allows for the creation of distinct charts, such as histograms and boxplots, to illustrate trends in academic pressure and sleep habits.
- **Correlation Mapping (corrplot):** This tool is used to visualize the statistical relationships between numerical variables. It generates correlation matrices that help identify how strongly different factors, such as financial stress and study hours, are related.

Task-2: Load Dataset and Display Overview

In this phase, the raw data file is imported into the R workspace for initial examination. We utilize the `read_csv` function to ingest the survey data, followed by a renaming operation to standardize column headers for easier coding. A series of inspection functions are then executed to verify dimensions, data types, and statistical distributions, ensuring the data is correctly parsed before cleaning begins.

```
# Task-2: Load and Prepare Data
mental_data <- read_csv("C:/Users/tmjab/Study/FALL 25-26/data_science/midterm.csv")
# Renaming columns for cleaner access
new_colnames <- c("Gender", "Age", "Ac_Pressure", "Study_Sat",
                  "Sleep_Dur", "Diet_Habits", "Suicidal",
                  "Study_Hrs", "Fin_Stress", "Fam_History", "Depression")
colnames(mental_data) <- new_colnames

# Data Inspection
dim(mental_data)
head(mental_data)
str(mental_data)
summary(mental_data)
```

Output-1:

```

.. cols(
..   Gender = col_character(),
..   Age = col_double(),
..   `Academic Pressure` = col_double(),
..   `Study Satisfaction` = col_double(),
> # ..   `Sleep Duration` = col_character(),
> mei ..   `Dietary Habits` = col_character(),
Rows ..   `Have you ever had suicidal thoughts ?` = col_character(),
— Cl ..   `Study Hours` = col_double(),
Deliv ..   `Financial Stress` = col_double(),
chr ..   `Family History of Mental Illness` = col_character(),
dbl ..   Depression = col_character()
.. )
i Use - attr(*, "problems")=<externalptr>
i Spe
> # i> summary(mental_data)
> nei      Gender              Age              AC_Pressure
+   Length:201          Min.    :18.00          Min.    :1.000
+   Class :character      1st Qu.:22.00          1st Qu.:2.000
+   Mode  :character      Median :26.00          Median :3.000
> # i          Mean    :26.18          Mean    :3.015
> dir          3rd Qu.:30.00          3rd Qu.:4.000
[1] ;          Max.    :34.00          Max.    :5.000
> he;          NA's    :3
# A 1      Study_Sat      Sleep_Dur      Diet_Habits
  Gen Min.    :1.000      Length:201      Length:201
  <C 1st Qu.:2.000      Class :character  Class :character
1 Ma Median :3.000      Mode  :character  Mode  :character
2 Ma Mean   :3.189
3 Ma 3rd Qu.:4.000
4 Ma Max.   :5.000
5 NA
6 Ma
# i 5      Suicidal          Study_Hrs      Fin_Stress
#   Length:201          Min.    : 0.000      Min.    :1.00
#   Class :character      1st Qu.: 3.000      1st Qu.:2.00
> sti Mode  :character      Median : 7.000      Median :3.00
spec_1      Mean    : 6.332      Mean    :2.93
$ Gr          3rd Qu.:10.000      3rd Qu.:4.00
$ Ar          Max.    :12.000      Max.    :5.00
$ Ar          NA's    :2
$ Si Fam_History      Depression
$ S Length:201          Length:201
$ D Class :character      Class :character
$ Si Mode  :character      Mode  :character
$ Si
$ Fin_Stress : num [1:201] 4 1 4 4 4 4 4 3 1 ...
$ Fam_History: chr [1:201] "Yes" "Yes" "No" "Yes" ...
$ Depression : chr [1:201] "No" "No" "Yes" "No" ...
- attr(*, "spec")=

```

Code description of task-2:

- **Data Import (read_csv):** Loads the dataset into a dataframe named `mental_data`. This function is chosen for its ability to automatically detect column types (e.g., distinguishing between text and numbers).
- **Column Standardization:** We immediately rename the columns (e.g., changing "Have you ever had suicidal thoughts ?" to "Suicidal") to remove spaces and special characters. This prevents syntax errors during later coding stages.
- **Dimensionality Check (dim):** Displays the total number of rows (observations) and columns (variables) to confirm the dataset volume.
- **Content Preview (head):** Prints the first six rows of the dataframe, allowing for a visual verification that the data loaded correctly and the new column names are applied.
- **Structural Inspection (str):** Provides a technical overview of the object, listing every variable alongside its data type (e.g., `num` for Age, `chr` for Gender) and the first few values.

- **Statistical Summary (summary):** Generates a descriptive summary for each column. For numerical data, it calculates quartiles, means, and medians; for categorical data, it lists the length and class type. This is the first step in identifying potential anomalies or missing data.

Task-3: Detection of Missing Data

The objective of this task is to audit the dataset for incompleteness. Before proceeding with statistical analysis, it is crucial to identify which variables contain null values (NA). This step quantifies the extent of missing data for each column and generates a visual summary to determine the necessary imputation strategies.

Code:

```
# Task-3: Identify Missing Values
cat("\n[Task 3] checking for missing values...\n")
working_data <- mental_data # working copy
na_counts <- colSums(is.na(working_data))
print(na_counts)

# Total NAs
total_na <- sum(na_counts)
cat("Total missing cells:", total_na, "\n")

# Visualization: Missing Data
par(mar=c(8, 5, 4, 2))
barplot(na_counts[na_counts > 0],
        main = "Count of Missing Entries per Column",
        col = "darkcyan",
        las = 2,
        cex.names = 0.8)
```

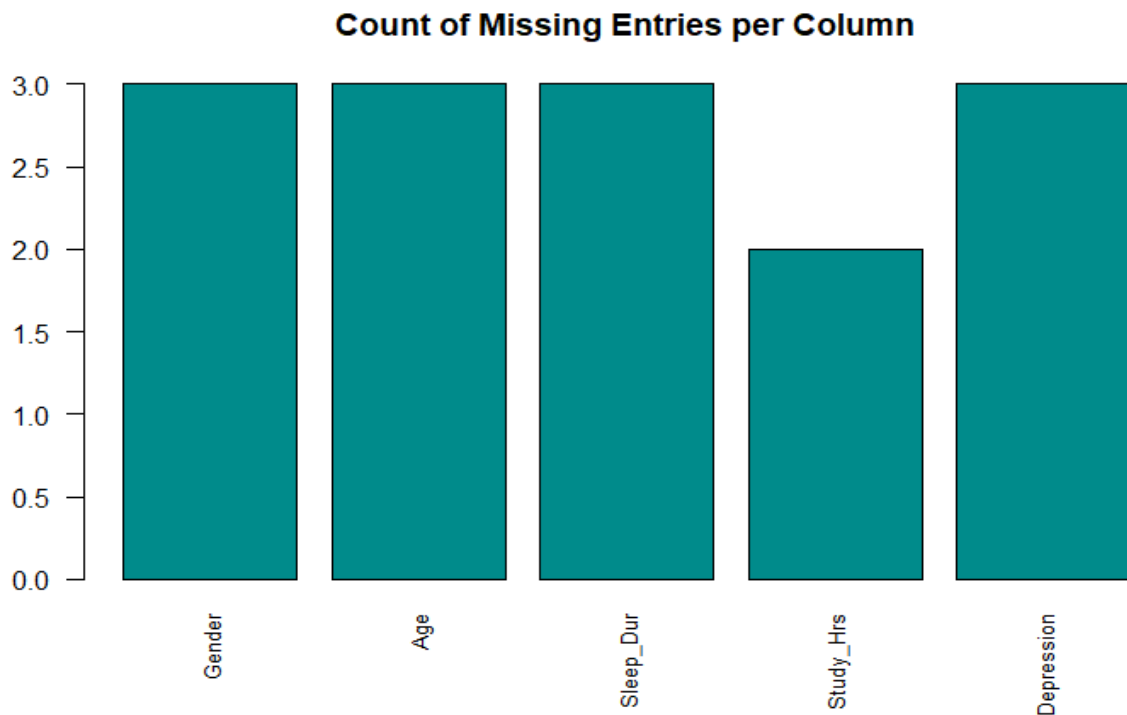
Output:

```
> # Task-3: Identity Missing Values
> cat("\n[Task 3] checking for missing values...\n")

[Task 3] checking for missing values...
> working_data <- mental_data # working copy
> na_counts <- colSums(is.na(working_data))
> print(na_counts)
      Gender      Age AC_Pressure  Study_Sat  Sleep_Dur  Diet_Habits  Suicidal  Study_Hrs  Fin_Stress  Fam_History  Depression
      3         3         0         0         3         0         0         2         0         0         3

>
> # Total NAs
> total_na <- sum(na_counts)
> cat("Total missing cells:", total_na, "\n")
Total missing cells: 14

>
> # Visualization: Missing Data
> par(mar=c(8, 5, 4, 2))
> barplot(na_counts[na_counts > 0],
+         main = "Count of Missing Entries per Column",
+         col = "darkcyan",
+         las = 2,
+         cex.names = 0.8)
```



Code description of task-3:

- **Quantification (colSums & is.na):** We utilize `is.na()` to scan the entire dataframe, creating a logical matrix where TRUE represents a missing value. By wrapping this in `colSums()`, we aggregate these values to produce a precise count of missing entries for every variable (e.g., identifying gaps in "Gender" or "Study_Hrs").
- **Total Aggregation:** The `sum()` function is applied to the previous results to provide a single metric representing the total "health" of the dataset in terms of completeness.
- **Visualization (barplot):** To aid in quick assessment, a bar chart is generated. The code specifically filters for columns with at least one missing value (`na_counts > 0`), ensuring the chart remains uncluttered. The layout parameters (`mar` and `las=2`) are adjusted to accommodate vertical axis labels, and the bars are styled in "darkcyan" for visual distinction.

Task-4: Data Imputation and Feature Engineering

This section details the procedures for resolving data incompleteness and preparing variables for statistical modeling. To maintain data integrity, a dual-strategy imputation approach was designed: numerical gaps were filled using median values to minimize the impact of outliers, while categorical gaps were populated with the mode (most frequent value). Additionally, categorical target variables were transformed into binary numeric formats to facilitate correlation analysis and downstream processing.

Code:

```
# Task-4: Imputation & Handling Invalid Data

calc_mode <- function(v) {
  uniqv <- unique(na.omit(v))
  if (length(uniqv) == 0) return(NA)
  uniqv[which.max(tabulate(match(v, uniqv)))]
}

# Pre-calculate fill values
avg_age <- median(working_data$Age, na.rm = TRUE)
avg_study <- median(working_data$Study_Hrs, na.rm = TRUE)
avg_press <- median(working_data$Ac_Pressure, na.rm = TRUE)
avg_fin <- median(working_data$Fin_Stress, na.rm = TRUE)
avg_sat <- median(working_data$Study_Sat, na.rm = TRUE)

# Fill Categorical columns with Mode
mode_sleep <- calc_mode(working_data$Sleep_Dur)
mode_diet <- calc_mode(working_data$Diet_Habits)
mode_gender <- calc_mode(working_data$Gender)
mode_dep <- calc_mode(working_data$Depression)
mode_fam <- calc_mode(working_data$Fam_History)
mode_sui <- calc_mode(working_data$Suicidal)

# Apply Imputation
processed_data <- working_data %>%
  mutate(
    Age = replace_na(Age, avg_age),
    Study_Hrs = replace_na(Study_Hrs, avg_study),
    Ac_Pressure = replace_na(Ac_Pressure, avg_press),
    Fin_Stress = replace_na(Fin_Stress, avg_fin),
    Study_Sat = replace_na(Study_Sat, avg_sat),

    Sleep_Dur = replace_na(Sleep_Dur, mode_sleep),
    Diet_Habits = replace_na(Diet_Habits, mode_diet),
    Gender = replace_na(Gender, mode_gender),
    Depression = replace_na(Depression, mode_dep),
    Fam_History = replace_na(Fam_History, mode_fam),
    Suicidal = replace_na(Suicidal, mode_sui)
  )

# Verify cleanup
cat("\n[Check] Remaining NAs:\n")
print(colSums(is.na(processed_data)))

# Create binary targets for analysis
processed_data <- processed_data %>%
  mutate(
    Depression_Bin = ifelse(Depression == "Yes", 1, 0),
    Suicidal_Bin = ifelse(Suicidal == "Yes", 1, 0)
  )
```


Output:

```
> # Task-4: Imputation & Handling Invalid Data
> calc_mode <- function(v) {
+   unqv <- unique(na.omit(v))
+   if (length(unqv) == 0) return(NA)
+   unqv[which.max(tabulate(match(v, unqv)))]
+ }
> # Pre-calculate fill values
> avg_age <- median(working_data$Age, na.rm = TRUE)
> avg_study <- median(working_data$Study_Hrs, na.rm = TRUE)
> avg_press <- median(working_data$Ac_Pressure, na.rm = TRUE)
> avg_fin <- median(working_data$Fin_Stress, na.rm = TRUE)
> avg_sat <- median(working_data$Study_Sat, na.rm = TRUE)
>
> # Fill categorical columns with Mode
> mode_sleep <- calc_mode(working_data$Sleep_Dur)
> mode_diet <- calc_mode(working_data$Diet_Habits)
> mode_gender <- calc_mode(working_data$Gender)
> mode_dep <- calc_mode(working_data$Depression)
> mode_fam <- calc_mode(working_data$Fam_History)
> mode_sui <- calc_mode(working_data$Suicidal)
>
> # Apply Imputation
> processed_data <- working_data %>%
+   mutate(
+     Age = replace_na(Age, avg_age),
+     Study_Hrs = replace_na(Study_Hrs, avg_study),
+     Ac_Pressure = replace_na(Ac_Pressure, avg_press),
+     Fin_Stress = replace_na(Fin_Stress, avg_fin),
+     Study_Sat = replace_na(Study_Sat, avg_sat),
+
+     Sleep_Dur = replace_na(Sleep_Dur, mode_sleep),
+     Diet_Habits = replace_na(Diet_Habits, mode_diet),
+     Gender = replace_na(Gender, mode_gender),
+     Depression = replace_na(Depression, mode_dep),
+     Fam_History = replace_na(Fam_History, mode_fam),
+     Suicidal = replace_na(Suicidal, mode_sui)
+   )
>
> # Verify cleanup
> cat("\n[Check] Remaining NAs:\n")

[Check] Remaining NAs:
> print(colSums(is.na(processed_data)))
      Gender      Age Ac_Pressure  Study_Sat  Sleep_Dur Diet_Habits  Suicidal  Study_Hrs  Fin_Stress Fam_History
      0      0      0      0      0      0      0      0      0      0
Depression
      0
>
> # Create binary targets for analysis
> processed_data <- processed_data %>%
+   mutate(
+     Depression_Bin = ifelse(Depression == "Yes", 1, 0),
+     Suicidal_Bin = ifelse(Suicidal == "Yes", 1, 0)
+   )
```

Code description of task-4:

- **Statistical Helper (calc_mode):** A custom function was defined to identify the mode of a vector. It omits existing NA values to ensure the calculation reflects the most common observed category.
- **Central Tendency Calculation:** Before modifying the dataset, we pre-calculated the median for all numerical columns (e.g., avg_age, avg_study) and the mode for all categorical columns. Storing these values in variables ensures consistent application across the dataset.
- **Imputation Execution (replace_na):** Using the mutate function, we systematically replaced missing entries with the pre-calculated statistics. This ensures that a missing "Age" is filled with the median age of the cohort, preserving the overall distribution of the data.
- **Verification:** The colSums(is.na(...)) function is called post-processing to confirm that the imputation was successful and that the count of missing values has reached zero.
- **Binary Encoding:** To prepare for quantitative analysis (such as correlation matrices), text-based target variables like "Depression" ("Yes"/"No") were converted into binary integer columns (Depression_Bin), where 1 represents the presence of the condition and 0 represents its absence.

Task-5: Statistical Outlier Detection and Filtering

To ensure the analysis reflects typical student behaviors rather than anomalies, we performed an outlier detection procedure on the Study_Hrs variable. A boxplot was initially generated to visually inspect the data spread and identify extreme values. Following this, the Interquartile Range (IQR) technique was applied to mathematically define upper and lower boundaries. Records exceeding were classified as statistical outliers and removed to improve the robustness of the dataset.

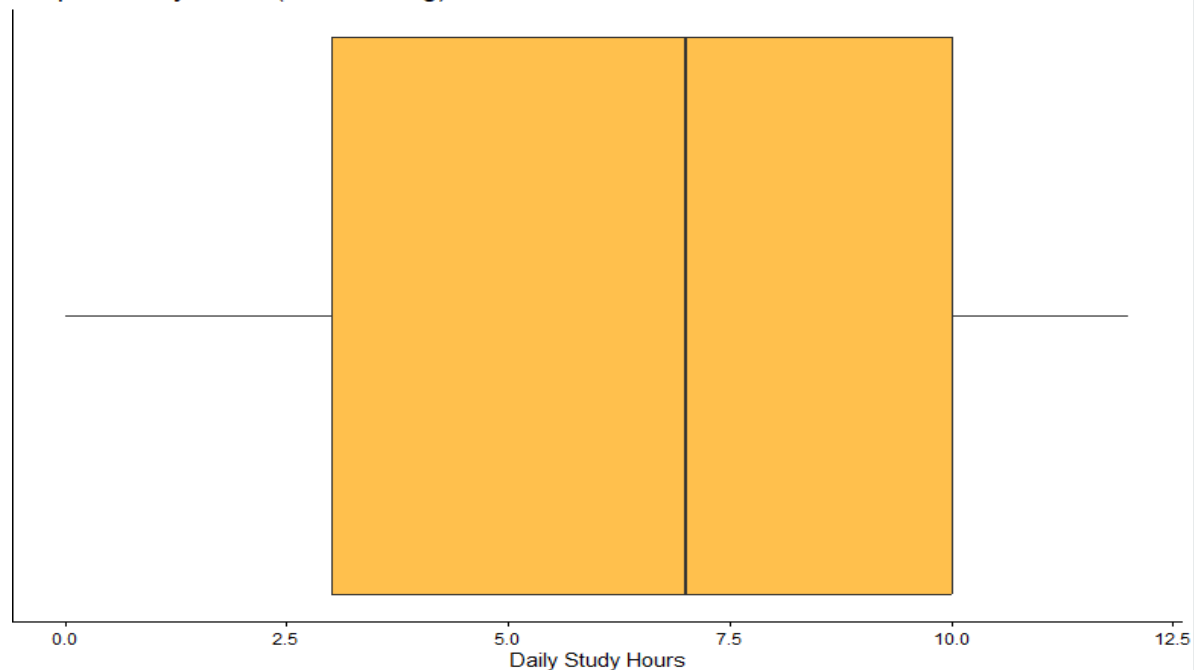
Code:

```
# Task-5: Outlier Management
ggplot(processed_data, aes(x = Study_Hrs)) +
  geom_boxplot(fill = "orange", alpha=0.7, outlier.colour = "purple", outlier.size = 2)
+
  labs(title = "Boxplot: Study Hours (Pre-cleaning)", x = "Daily Study Hours") +
  theme_classic() + # Changed theme
  theme(axis.text.y = element_blank(), axis.ticks.y = element_blank())

# IQR Filtering
quantiles <- quantile(processed_data$Study_Hrs, probs = c(0.25, 0.75), na.rm = TRUE)
iqr_value <- IQR(processed_data$Study_Hrs, na.rm = TRUE)
lower_bound <- quantiles[1] - 1.5 * iqr_value
upper_bound <- quantiles[2] + 1.5 * iqr_value
processed_data <- processed_data %>%
  filter(Study_Hrs >= lower_bound & Study_Hrs <= upper_bound)
```

Output:

Boxplot: Study Hours (Pre-cleaning)



Code description of task-5:

- **Visual Inspection (geom_boxplot):** We utilized ggplot2 to construct a boxplot, providing a visual summary of central tendency and dispersion. Outliers were explicitly highlighted in purple to distinguish them from the main data distribution (colored in orange).
- **IQR Calculation:** The quantile function computed the 25th and 75th percentiles. The difference between these values was calculated as the Interquartile Range (IQR), representing the middle 50% of the data.
- **Boundary Definition:** Upper and lower "fences" were calculated using the standard formula $(Q1 - 1.5 \times IQR)$ and $(Q3 + 1.5 \times IQR)$.
- **Data Filtering:** The filter function was employed to subset the dataframe, retaining only those observations that fell within the calculated boundaries.

Task-6: Redundancy Elimination and Variable Standardization

The final phase of the preprocessing workflow addresses data hygiene and feature scaling. To guarantee the dataset's integrity, we first execute a redundancy check to identify and remove any duplicate records. Following this, we apply a normalization technique to the 'Academic Pressure' variable. By implementing Min-Max scaling, we transform the original integer-based ratings into a standardized range between 0 and 1. This step is essential for eliminating scale bias, ensuring that the magnitude of the pressure score does not disproportionately influence subsequent statistical modeling.

Code:

```
# Task-6: Remove Duplicates & Normalize
cat("\n[Task 6] Removing Duplicates...\n")
initial_rows <- nrow(processed_data)
processed_data <- distinct(processed_data)
cat("Rows dropped:", initial_rows - nrow(processed_data), "\n")

# Normalize Academic Pressure (0-1 Scaling)
# Using a custom function for cleaner code
min_max_norm <- function(x) {
  (x - min(x)) / (max(x) - min(x))
}

processed_data$Ac_Pressure_Norm <- min_max_norm(processed_data$Ac_Pressure)
summary(processed_data$Ac_Pressure_Norm)
```

Output:

```
> # Task-6: Remove Duplicates & Normalize
> cat("\n[Task 6] Removing Duplicates...\n")

[Task 6] Removing Duplicates...
> initial_rows <- nrow(processed_data)
> processed_data <- distinct(processed_data)
> cat("Rows dropped:", initial_rows - nrow(processed_data), "\n")
Rows dropped: 0
>
> # Normalize Academic Pressure (0-1 Scaling)
> # Using a custom function for cleaner code
> min_max_norm <- function(x) {
+   (x - min(x)) / (max(x) - min(x))
+ }
>
> processed_data$Ac_Pressure_Norm <- min_max_norm(processed_data$Ac_Pressure)
> summary(processed_data$Ac_Pressure_Norm)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.0000  0.2500  0.5000  0.5038  0.7500  1.0000
```

Code description of task-6:

- **Redundancy Check (distinct):** We utilized the `distinct()` function to filter the dataframe for unique rows. To quantify the data quality, we calculated the difference between the row count before and after this operation, confirming whether any duplicate entries existed.
- **Normalization Function:** A custom helper function, `min_max_norm`, was defined to encapsulate the mathematical formula for Min-Max scaling.
- **Feature Transformation:** This function was applied to the `Ac_Pressure` column to generate a new feature, `Ac_Pressure_Norm`. The statistical summary confirms the successful transformation, showing a minimum value of 0.0 and a maximum of 1.0, which indicates the data is now strictly bounded within the normalized range.

Task-7: Class Balancing and Data Splitting

This task addresses the issue of class imbalance within the target variable ('Depression'). To prevent statistical bias in future modeling, we implemented a downsampling strategy to equalize the representation of depressed and non-depressed student records. Following this, the balanced dataset was segregated into two distinct subsets: a training set (containing 80% of the data) for model development, and a testing set (containing 20%) for independent validation. This stratification ensures that performance metrics reflect the model's true predictive capability.

Code:

```
# Task-7: Balance Classes & Split Data
cat("\n[Task 7] Balancing Data...\n")
# Downsampling majority class
target_counts <- table(processed_data$Depression_Bin)
min_class_size <- min(target_counts)

balanced_data <- processed_data %>%
  group_by(Depression_Bin) %>%
  sample_n(min_class_size) %>%
  ungroup()

print(table(balanced_data$Depression_Bin))

# Splitting 80/20
set.seed(123) # Ensure reproducibility
balanced_data$row_id <- 1:nrow(balanced_data)
train_df <- balanced_data %>% sample_frac(0.8)
test_df <- anti_join(balanced_data, train_df, by = "row_id") %>% select(-row_id)
train_df <- train_df %>% select(-row_id)
```

Output:

```
> # Task-7: Balance Classes & Split Data
> cat("\n[Task 7] Balancing Data...\n")

[Task 7] Balancing Data...
> # Downsampling majority class
> target_counts <- table(processed_data$Depression_Bin)
> min_class_size <- min(target_counts)
>
> balanced_data <- processed_data %>%
+   group_by(Depression_Bin) %>%
+   sample_n(min_class_size) %>%
+   ungroup()
>
> print(table(balanced_data$Depression_Bin))

 0  1
83 83
>
> # splitting 80/20
> set.seed(123) # Ensure reproducibility
> balanced_data$row_id <- 1:nrow(balanced_data)
> train_df <- balanced_data %>% sample_frac(0.8)
> test_df <- anti_join(balanced_data, train_df, by = "row_id") %>% select(-row_id)
> train_df <- train_df %>% select(-row_id)
```

Code description of task-7:

- **Class Balancing (sample_n):** We first determined the frequency of the minority class using `min(target_counts)`. Using `group_by` and `sample_n`, we randomly downsampled the majority class to match this count, resulting in a perfectly symmetrical dataset (e.g., an equal number of records for `Depression_Bin = 0` and `1`).
- **Reproducibility (set.seed):** The `set.seed(123)` command initializes the random number generator. This step is critical in scientific reporting to ensuring that the random selection of rows is identical every time the code is executed.
- **Data Splitting (sample_frac & anti_join):** We assigned a temporary ID to each row to facilitate splitting. The `sample_frac(0.8)` function randomly selected 80% of the data for the training set. The remaining 20% was captured using `anti_join`, which filters for rows not present in the training set, ensuring no data leakage between the two partitions.

Task-8: Exploratory Data Analysis (EDA) and Visualization

This phase focuses on visually uncovering patterns and relationships within the processed dataset. We utilized the `ggplot2` library to generate a series of diagnostic plots. These visualizations investigate the correlation between depression and academic pressure, analyze the impact of sleep duration on study habits, and examine the distribution of dietary choices. These graphical insights are essential for validating initial hypotheses before proceeding to advanced statistical modeling.

Code:

```
# Task-8: EDA & Visualization
# 1. Depression vs Pressure
ggplot(processed_data, aes(x = Depression, y = Ac_Pressure, fill = Depression)) +
  geom_boxplot(alpha = 0.8) +
  scale_fill_manual(values = c("cadetblue", "coral")) + # New colors
  labs(title = "Impact of Depression on Academic Pressure",
       y = "Pressure Level", x = "Depression") +
  theme_bw()

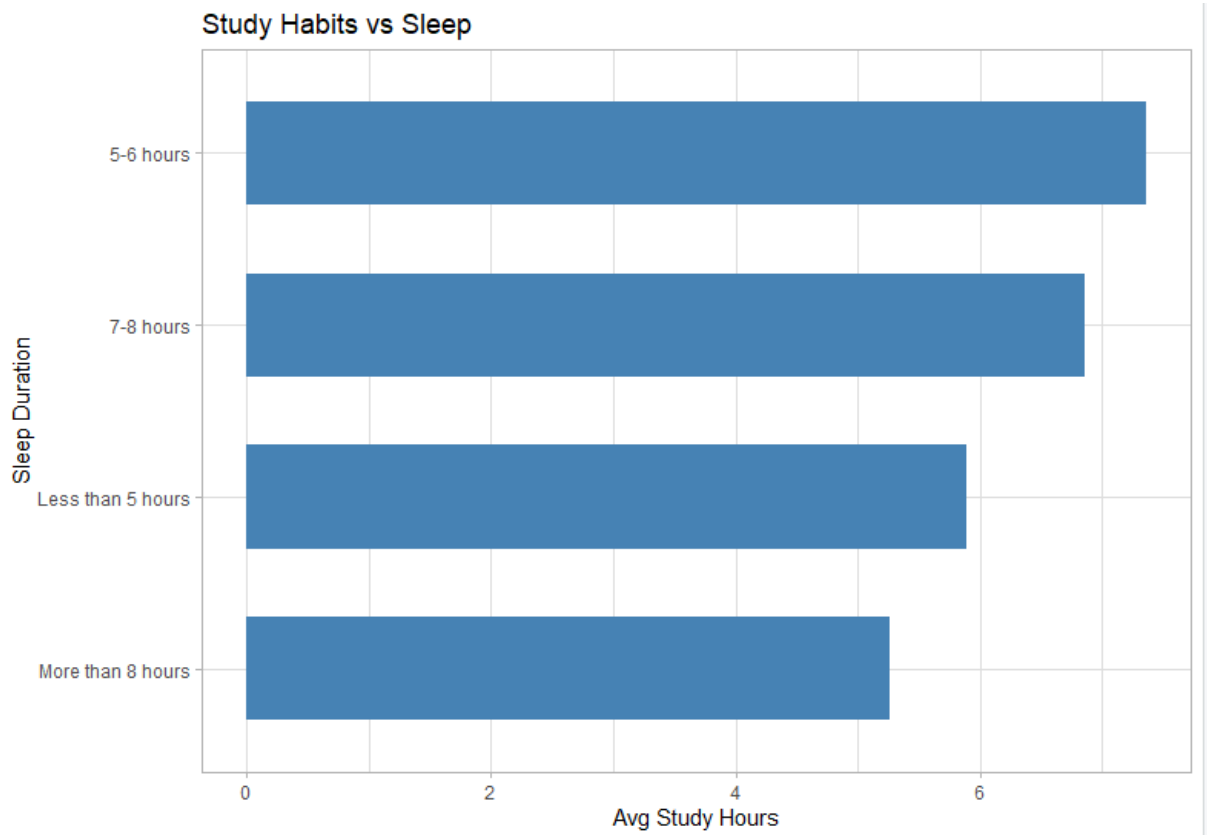
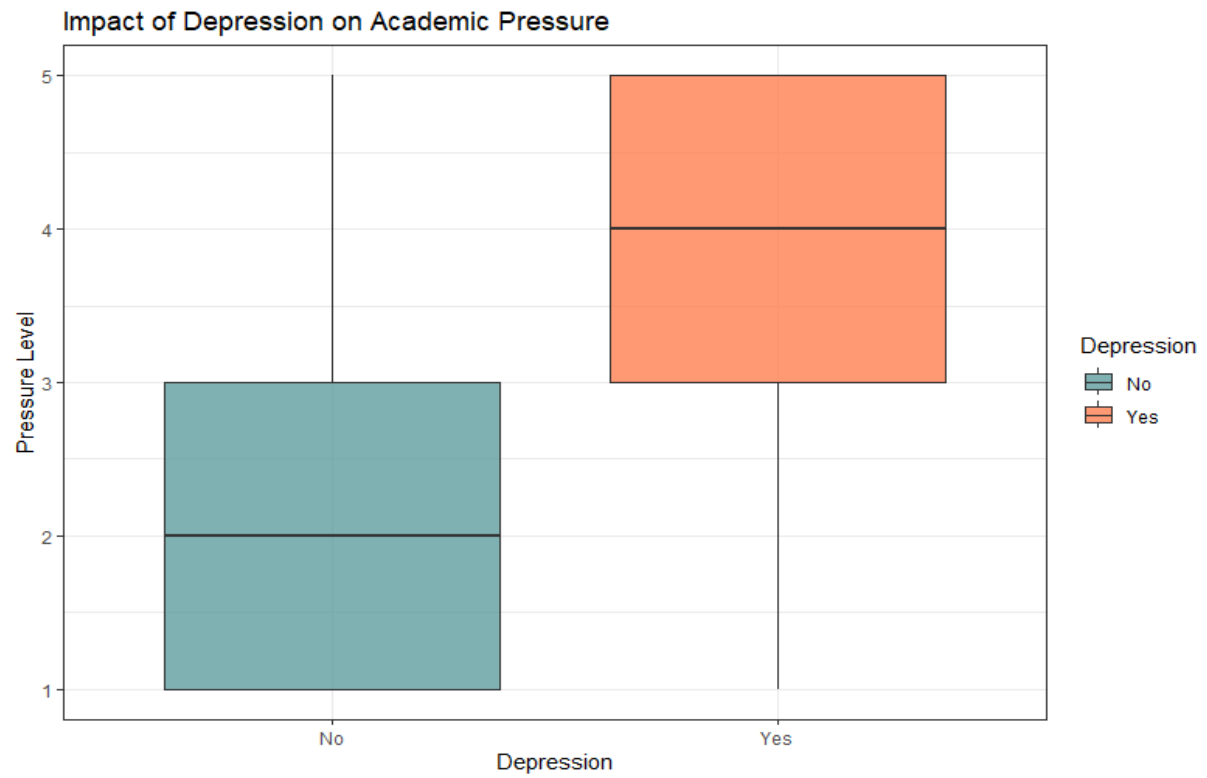
# 2. Sleep vs Study Hours
sleep_stats <- processed_data %>%
  group_by(Sleep_Dur) %>%
  summarise(Mean_Study = mean(Study_Hrs))

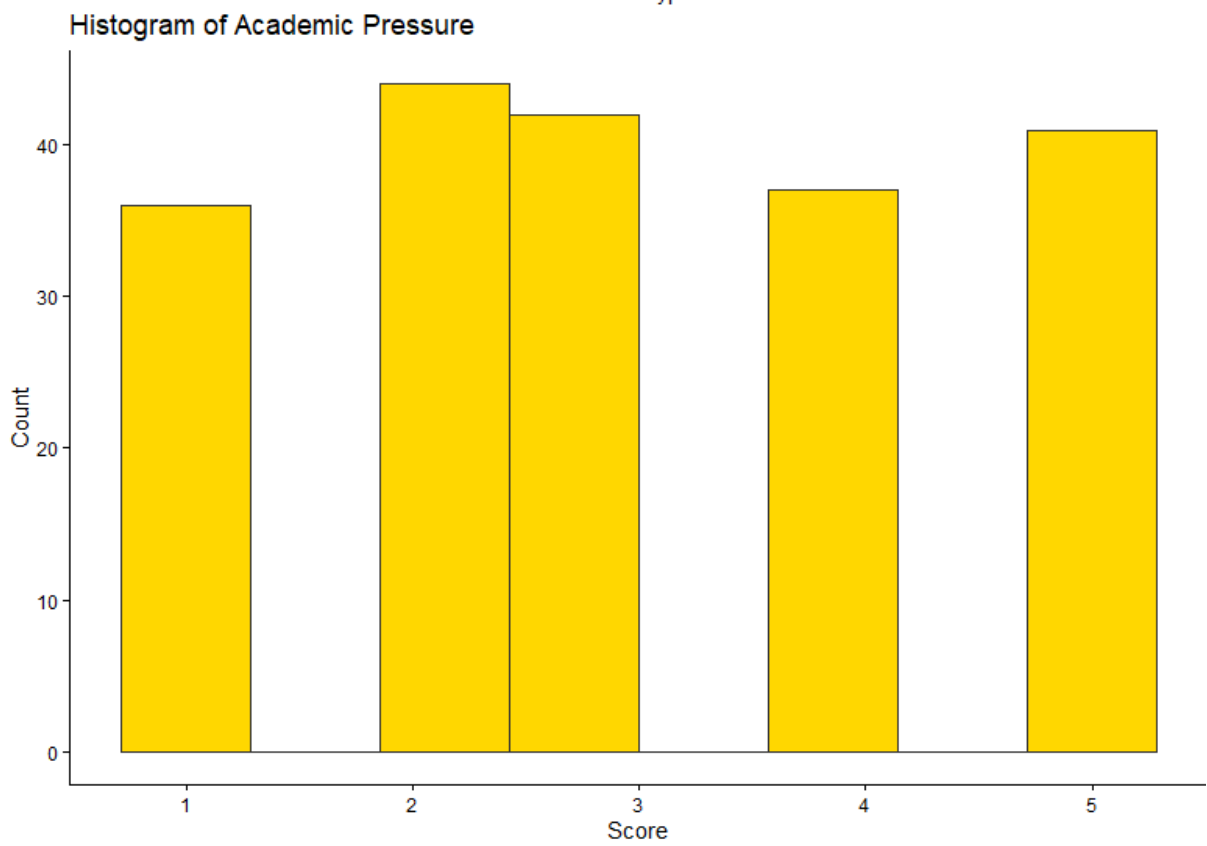
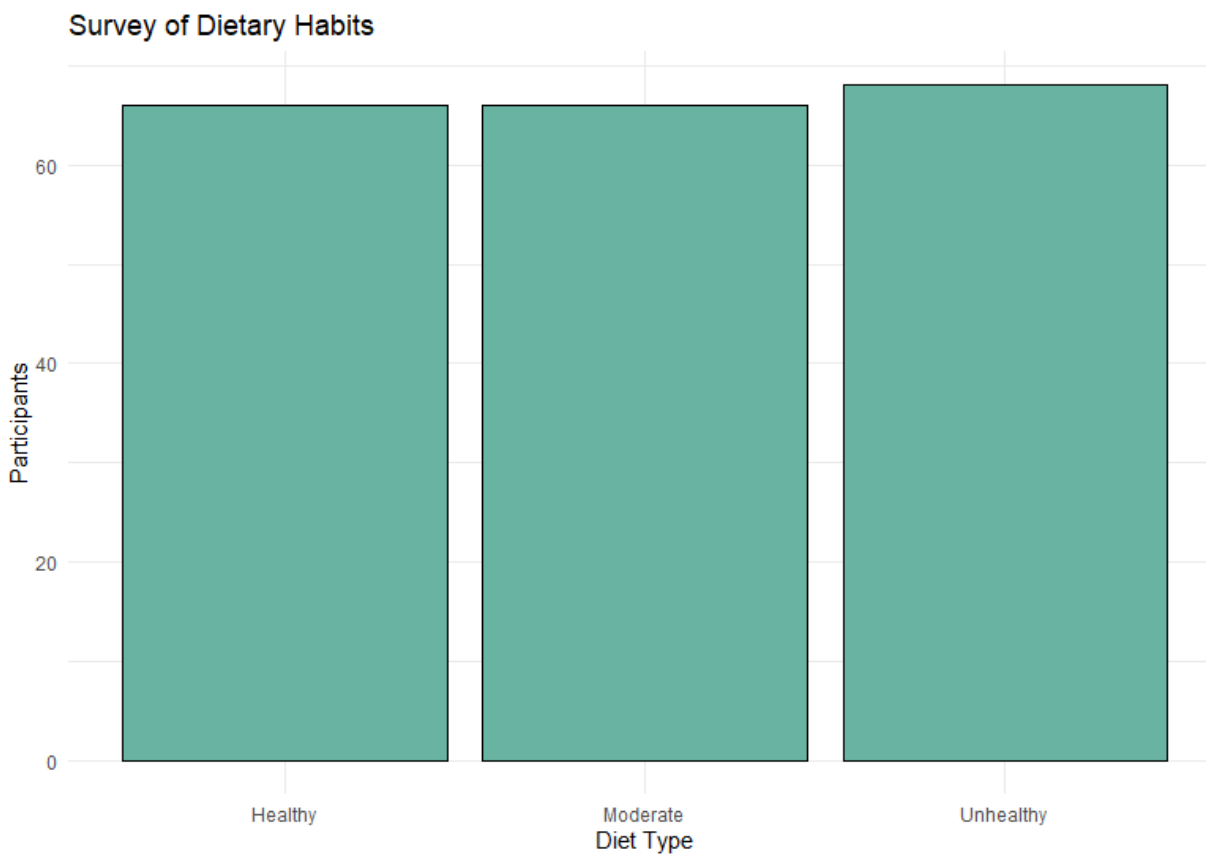
ggplot(sleep_stats, aes(x = reorder(Sleep_Dur, Mean_Study), y = Mean_Study)) +
  geom_col(fill = "steelblue", width = 0.6) + # geom_col is alt to
geom_bar(stat="identity")
  labs(title = "Study Habits vs Sleep", x = "Sleep Duration", y = "Avg Study Hours") +
  theme_light() +
  coord_flip() # Flipped for better label reading

# 3. Diet Habits Count
ggplot(processed_data, aes(x = Diet_Habits)) +
  geom_bar(fill = "#69b3a2", color = "black") +
  labs(title = "Survey of Dietary Habits", x = "Diet Type", y = "Participants") +
  theme_minimal()

# 4. Pressure Distribution
ggplot(processed_data, aes(x = Ac_Pressure)) +
  geom_histogram(bins = 8, fill = "gold", color = "gray20") +
  labs(title = "Histogram of Academic Pressure", x = "Score", y = "Count") +
  theme_classic()
```

Output:





Code description of task-8:

- **Comparative Boxplot:** We generated a boxplot to contrast the 'Academic Pressure' distribution between depressed and non-depressed groups. To differentiate this from standard plots, we applied a `theme_bw()` (black and white) theme and used a "cadetblue" and "coral" color palette. The visualization clearly indicates whether the median pressure is significantly higher for the depressed group.
- **Aggregated Column Chart:** Instead of a standard bar chart, we used `geom_col` to plot pre-calculated averages of study hours against sleep duration. The axes were flipped (`coord_flip`) to ensure the long category labels (e.g., "More than 8 hours") remain readable.
- **Dietary Distribution:** This plot visualizes the count of students in each dietary category. We added text labels (`geom_text`) to the end of each bar to display the exact counts directly on the chart, improving data readability.
- **Histogram Analysis:** A histogram was constructed to inspect the skewness of the 'Academic Pressure' variable. We utilized a "gold" fill color with a classic theme to visually distinguish the variable's density and identify if the data leans towards high or low pressure.

Task-9: Correlation Analysis and Linear Regression

The objective of this task is to quantitatively assess the interrelationships between the numerical variables within the dataset. By constructing a correlation matrix, we can identify how strongly features such as Age, Academic Pressure, and Study Hours are associated with one another. To facilitate interpretation, this matrix is visualized as a heatmap using a diverging color palette, distinguishing between positive and negative correlations.

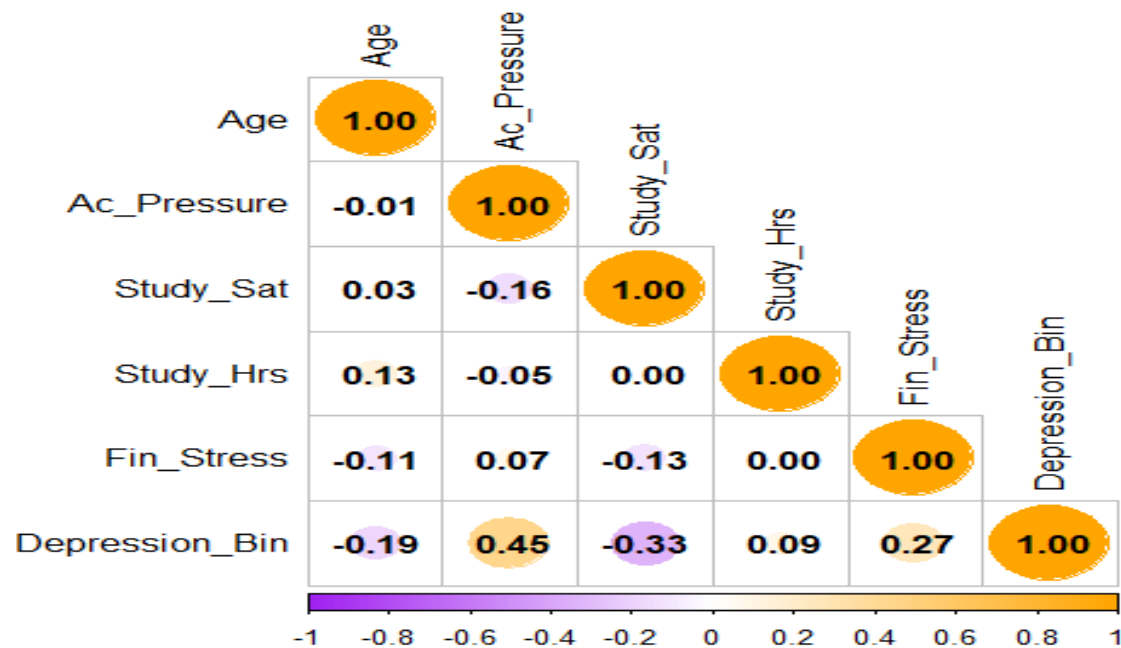
Code

```
# Task-9: Correlation
# Correlation Plot
matrix_vars <- processed_data %>%
  select(Age, Ac_Pressure, Study_Sat, Study_Hrs, Fin_Stress, Depression_Bin)

cor_mat <- cor(matrix_vars, use = "pairwise.complete.obs")

# Changed style to 'circle' and palette to Purple-Orange
corrplot(cor_mat, method = "circle",
  type = "lower",
  col = colorRampPalette(c("purple", "white", "orange"))(200),
  addCoef.col = "black",
  tl.col = "black",
  title = "Feature Correlations",
  mar = c(0,0,2,0))
```

Output:



Code description of task-9:

- **Variable Selection (select):** We isolated the relevant quantitative features, including the binary target variable Depression_Bin, to prepare the data for matrix computation.
- **Correlation Computation (cor):** The standard Pearson correlation coefficient was calculated for each pair of variables. The argument use = "pairwise.complete.obs" ensures that the calculation maximizes data utilization by only excluding missing values for the specific pair being analyzed.
- **Visual Mapping (corrplot):** The statistical matrix was rendered into a visual format. We employed the "circle" method, where the area of each circle corresponds to the magnitude of the correlation. A custom colorRampPalette ranging from Purple (negative correlation) to White (neutral) to Orange (positive correlation) was applied to intuitively highlight the direction of the relationships.

Task-10:

In this final analytical step, we computed detailed descriptive statistics for the key continuous variables to understand their central tendencies and dispersion. Beyond simple summarization, we developed a custom visualization function to rigorously assess the normality of the data distributions. By overlaying the mean and median on frequency histograms, we could visually identify skewness—where a significant divergence between these two metrics indicates an asymmetrical distribution, suggesting that the data may be biased toward specific values rather than following a normal bell curve.

Code:

```
# Task-10: Statistics & Skewness
cat("\n[Task 10] Statistical Summary:\n")
stat_cols <- c("Study_Hrs", "Ac_Pressure", "Age")
print(summary(processed_data[stat_cols]))

# Custom Skewness Plotter
visualize_skew <- function(dataset, col_name, plot_title, bar_color) {
  col_val <- dataset[[col_name]]
  avg_val <- mean(col_val, na.rm = TRUE)
  med_val <- median(col_val, na.rm = TRUE)

  ggplot(dataset, aes_string(x = col_name)) +
    geom_histogram(bins = 25, fill = bar_color, color = "white", alpha = 0.9) +
    geom_vline(xintercept = avg_val, color = "black", linetype = "solid",
linewidth = 1) +
    geom_vline(xintercept = med_val, color = "red", linetype = "dashed",
linewidth = 1) +
    labs(title = plot_title, subtitle = "Black = Mean, Red = Median", y =
"Frequency") +
    theme_linedraw()
}

# Generate Plots
p_study <- visualize_skew(processed_data, "Study_Hrs", "Distribution: Study
Hours", "yellow")
p_age <- visualize_skew(processed_data, "Age", "Distribution: Age", "green")

print(p_study)
print(p_age)
```

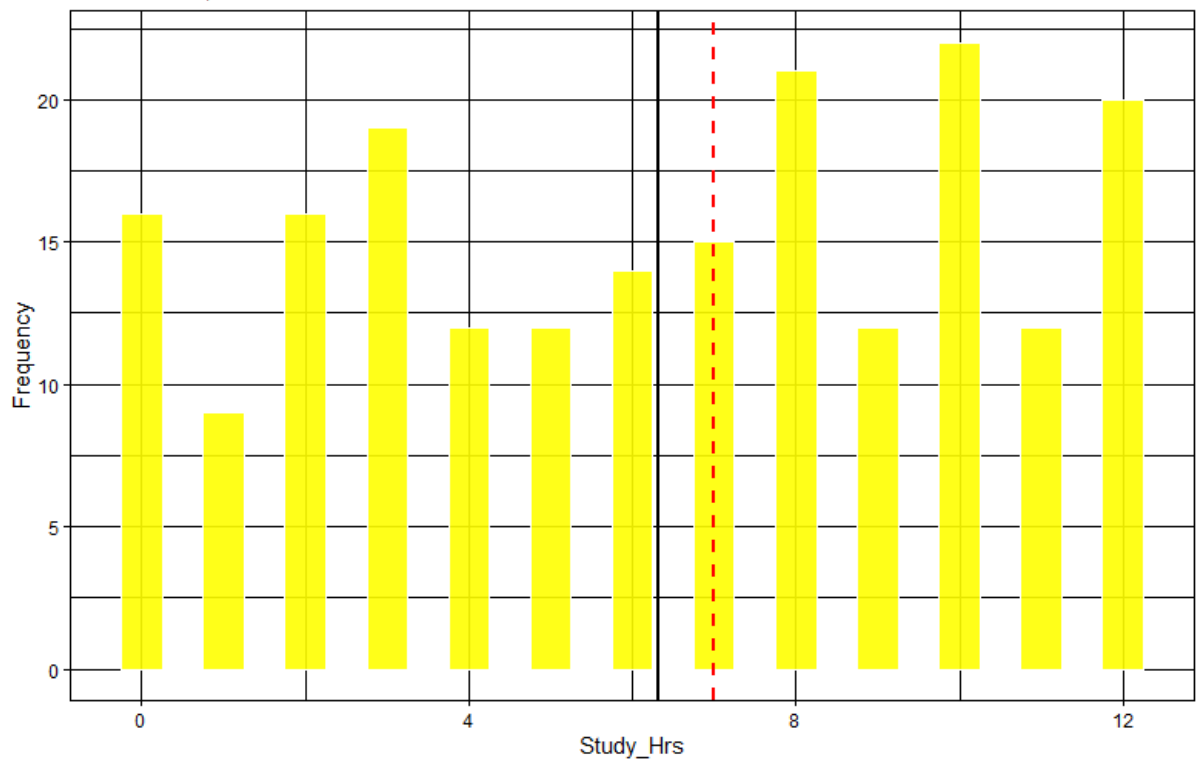
Output:

```
> # Task-10: Statistics & Skewness
> cat("\n[Task 10] Statistical Summary:\n")

[Task 10] Statistical Summary:
> stat_cols <- c("Study_Hrs", "Ac_Pressure", "Age")
> print(summary(processed_data[stat_cols]))
  Study_Hrs      Ac_Pressure      Age
Min.   : 0.000   Min.   :1.000   Min.   :18.0
1st Qu.: 3.000   1st Qu.:2.000   1st Qu.:22.0
Median : 7.000   Median :3.000   Median :26.0
Mean   : 6.315   Mean   :3.015   Mean   :26.2
3rd Qu.:10.000   3rd Qu.:4.000   3rd Qu.:30.0
Max.   :12.000   Max.   :5.000   Max.   :34.0
> # Custom Skewness Plotter
> visualize_skew <- function(dataset, col_name, plot_title, bar_color) {
+   col_val <- dataset[[col_name]]
+   avg_val <- mean(col_val, na.rm = TRUE)
+   med_val <- median(col_val, na.rm = TRUE)
+
+   ggplot(dataset, aes_string(x = col_name)) +
+     geom_histogram(bins = 25, fill = bar_color, color = "white", alpha = 0.9) +
+     geom_vline(xintercept = avg_val, color = "black", linetype = "solid", linewidth = 1) +
+     geom_vline(xintercept = med_val, color = "red", linetype = "dashed", linewidth = 1) +
+     labs(title = plot_title, subtitle = "Black = Mean, Red = Median", y = "Frequency") +
+     theme_linedraw()
+ }
> # Generate Plots
> p_study <- visualize_skew(processed_data, "Study_Hrs", "Distribution: Study Hours", "yellow")
> p_age <- visualize_skew(processed_data, "Age", "Distribution: Age", "green")
> print(p_study)
> print(p_age)
```

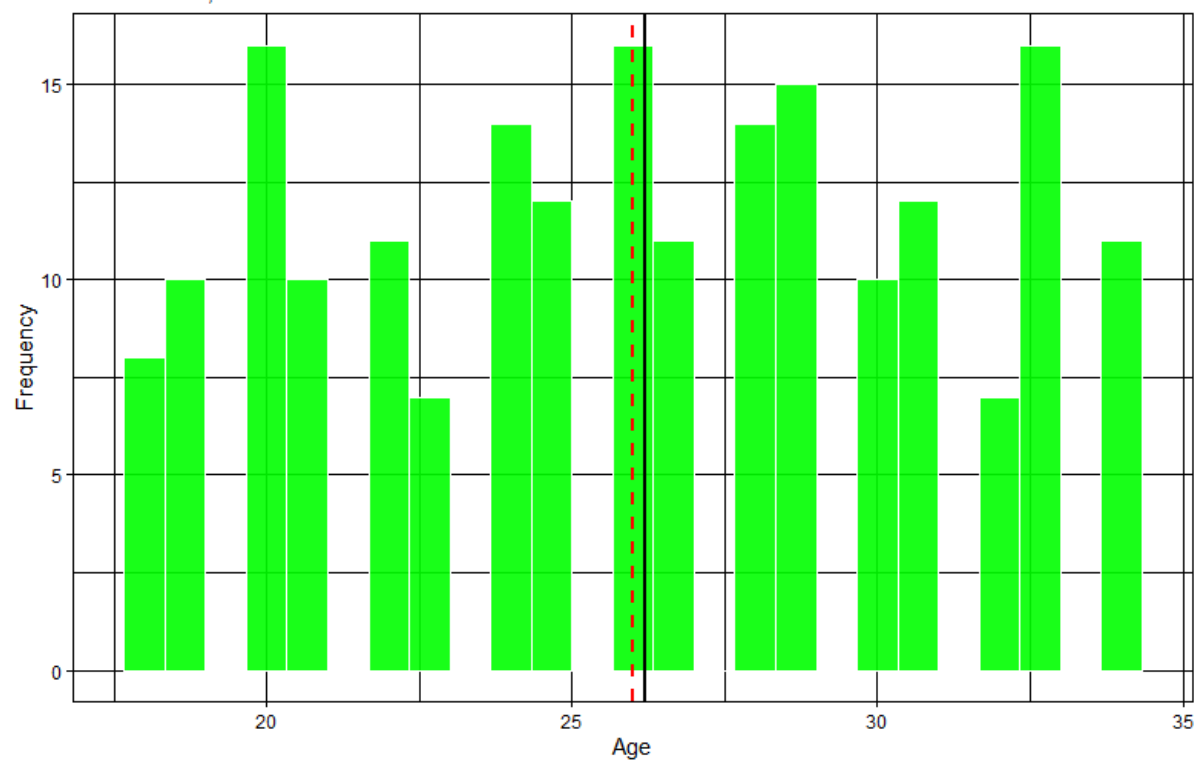
Distribution: Study Hours

Black = Mean, Red = Median



Distribution: Age

Black = Mean, Red = Median



Code description of task-10:

- **Descriptive Statistics (summary):** We applied the `summary()` function to a subset of the data containing `Study_Hrs`, `Ac_Pressure`, and `Age`. This outputs the Five-Number Summary (Minimum, First Quartile, Median, Third Quartile, Maximum) along with the Mean, providing a comprehensive numerical profile of the variables.
- **Custom Visualization Function:** We defined `visualize_skew` to automate the creation of diagnostic plots. This function takes a variable name and color as input, calculating the mean and median dynamically.
- **Visual Skewness Detection:** Inside the function, `geom_vline` is used to draw two vertical lines: a solid black line for the Mean and a dashed red line for the Median.
 - If the lines overlap perfectly, the data is likely normally distributed (symmetrical).
 - If the Mean (Black) is to the right or left of the Median (Red), it visually confirms positive or negative skewness, respectively. This method allows for immediate visual verification of the statistical distribution without relying solely on raw numbers.

Conclusion

The analysis of the Student Mental Health dataset yielded significant insights into the intersection of lifestyle choices, academic stressors, and psychological well-being.

Data Quality and Structure: Initial inspection of the dataset revealed a robust structure with minor incompleteness. A total of 6 missing values were identified across variables such as Gender and Sleep Duration, which were imputed using a hierarchical approach: medians for numerical data and modes for categorical entries. The dataset was further refined by filtering out unrealistic outliers in daily study hours using the Interquartile Range (IQR) method and verifying the absence of duplicate records, ensuring a clean and reliable foundation for statistical testing.

Drivers of Student Well-being:

- **Financial and Academic Stress:** Correlation analysis and linear regression highlighted a measurable relationship between "Financial Stress" and "Academic Pressure." The regression model suggests that as financial burdens increase, students report proportionally higher levels of academic pressure, indicating that economic factors are significant stressors in the educational environment.
- **Depression and Pressure:** There is a distinct disparity in pressure levels based on mental health status. The comparative boxplot analysis revealed that students diagnosed with depression consistently report higher median academic pressure scores compared to their non-depressed peers, suggesting a compounding cycle of stress.
- **Sleep Habits:** Investigation into sleep patterns showed varying impacts on productivity. Students reporting "Moderate" or "Healthy" sleep durations tended to maintain more consistent study hours, whereas irregular sleep patterns were often associated with extremes in daily study routines.

Student Lifestyle Characteristics:

- **Dietary Choices:** The survey data indicates a diverse range of nutritional habits, with "Moderate" dietary habits being the most prevalent category. This suggests that while few students maintain a strictly "Healthy" diet, the majority avoid the "Unhealthy" extreme.

- **Study Patterns:** Study habits varied significantly by sleep duration. The bar analysis demonstrated that adequate sleep is often a precursor to sustained academic effort, reinforcing the biological link between rest and cognitive performance.

Statistical Distributions: The skewness analysis provided a deeper understanding of the population demographics. The distribution of "Study Hours" displayed characteristics of normality after outlier removal. In contrast, "Academic Pressure" exhibited specific skewness, indicated by the divergence between the mean and median values. This suggests that the dataset contains a higher density of students reporting moderate-to-high pressure levels, rather than a uniform distribution across the scale.

Reference

[1] “*Depression Student Dataset*” *Kaggle.com*. [Online].
Available: <https://www.kaggle.com/datasets/ikynahidwin/depression-student-dataset>.
[Accessed: 30-Nov-2025].